



# Navi 个人导航站

拉取镜像部署指南

无需构建 · 无需源码 · 双架构 · 一条 pull 命令搞定

适用环境: 任意 Linux + Docker (x86\_64 / arm64)

镜像地址: ghcr.io/mijunyi/navi-homepage

文档日期: 2026 年 9 月 19 日

## 目录

1. 这份指南解决什么问题

- 2. 部署前必读：三条硬规则
- 3. 前提一：镜像已经存在（如何发布第一个版本）
- 4. 前提二：包可见性与登录凭据
- 5. 前提三：网络能到得了镜像仓库
- 6. 前提四：CPU 架构与容器运行时
- 7. 部署步骤（命令行，推荐）
- 8. 部署步骤（图形界面 / NAS）
- 9. 首次访问与验证
- 10. 升级、回滚与固定版本
- 11. 常见问题与排查
- 12. 部署检查清单
- 13. 命令速查

## 1. 这份指南解决什么问题

本项目有两种拿到镜像的方式，本指南只讲第二种（更省事的那种）：

| 方式                               | 目标机器要做什么                                                             | 适合                        |
|----------------------------------|----------------------------------------------------------------------|---------------------------|
| 源码构建<br>docker-compose.yml       | 拿到源码 → <code>docker build</code> （需联网拉 <code>node:22-alpine</code> ） | 你想改代码、或镜像源被墙得厉害           |
| 拉取镜像<br>docker-compose.image.yml | 只需 <code>docker pull</code>                                          | 不想装构建环境、不想传源码、要在多台机器上重复部署 |

镜像由 **GitHub Actions** 在打 tag 时自动构建并推送到 GitHub 的镜像仓库 **GHCR**（[ghcr.io](https://ghcr.io)），同时产出 `linux/amd64` 与 `linux/arm64` 两种架构。整个构建过程跑在 GitHub 的机器上，你本地不需要装 **Docker**。

一个必须先澄清的概念：代码仓库 [github.com/mijunyi/navi-homepage](https://github.com/mijunyi/navi-homepage) 不是镜像仓库，不能直接 `docker pull`。GitHub 提供的镜像托管是另一个服务 [ghcr.io](https://ghcr.io)。源码推上去了 ≠ 镜像就有了——镜像需要额外发布一次（见第 3 节）。

## 2. 部署前必读：三条硬规则

下面三点如果忽略，会分别导致「拉不到镜像」「容器起不来」「站点裸奔」。

| 硬规则                                     | 原因与做法                                                                                                   |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------|
| ① 先确保 <code>data/config.json</code> 已存在 | 编排文件挂载的是整个 <code>./data</code> 目录，这会遮住镜像内置的示例配置。宿主机上若没有 <code>data/config.json</code> ，服务启动时会因读不到配置而失败。 |

|               |                                                                                                                                                                       |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|               | ✅ 第 7 节的第 1 步就是从镜像里把示例配置导出来，先做这一步。                                                                                                                                    |
| ② 必须提供访问密码    | 编排文件用的是 <code>\${NAVI_PASSWORD:?...}</code> 强制校验：缺少该变量时 <b>compose 会直接报错退出</b> 。这是刻意设计——避免静默退化成「无密码开放访问」（任何拿到地址的人都能进编辑模式改你的配置）。<br>做法：在编排文件同目录放一份 <code>.env</code> 。 |
| ③ 升级时要显式 pull | Docker 默认会复用本地已有镜像。up -d 不会自动去远端核对 latest， <b>你会在不知情的情况下一直跑旧版本</b> 。升级请先 <code>docker compose ... pull</code> ，或固定版本号（见第 10 节）。                                       |

**数据安全：**镜像只是程序，随时可以换；真正不可再生的只有 data/ 目录（config.json + 上传的图片）。升级、重建容器都不会动它。升级前建议先备份：`tar czf navi-data-$(date +%F).tar.gz -C ./data .`

### 3. 前提一：镜像已经存在（如何发布第一个版本）

镜像由仓库里的工作流 `.github/workflows/docker-publish.yml` 构建。它有三个触发入口，对应三种不同的产物：

| 触发方式                         | 产出的标签            | 用途                                     |
|------------------------------|------------------|----------------------------------------|
| 推送 main 分支                   | edge             | 开发中的最新代码， <b>不会动 latest</b> ，避免污染别人的部署 |
| 打 tag，形如 v1.2.3              | 1.2.3、1.2、latest | 正式发布。只有这条路径会更新 latest                  |
| Actions 页面手动触发（Run workflow） | 同 main → edge    | 不需要改代码，只想立刻构建一次                        |

如果现在去 pull 报 manifest unknown 或 not found，通常就是「镜像还没发布过」。按下面任一方式让工作流先跑一次：

**A 手动触发一次（最快，不需要动代码）**

- 1. 打开 `https://github.com/mijunyi/navi-homepage/actions`
- 2. 左栏选「构建并发布镜像（GHCN）」
- 3. 右上角 **Run workflow** → 分支选 main → 点绿色按钮
- 4. 等 3~8 分钟（首次要构建两种架构，之后有缓存会快很多）
- 5. 看到两个作业都打勾即可，此时 ghcr.io/mijunyi/navi-homepage:edge 已经可以拉取

**B 打一个正式版本标签**

```
# 在源码仓库里执行（本项目已配好 SSH 通道）
git tag v1.0.0
```

```
git push origin v1.0.0
```

推送后工作流会自动构建并产出 1.0.0 / 1.0 / latest 三个标签。

**workflows里的冒烟验证：**构建完成后还有一个独立作业会 **真实启动一次容器**，校验镜像架构、暴露端口、声明数据卷，并请求 /api/health 确认返回 "ok":true。所以「作业全绿」意味着这个镜像确实跑得起来，而不只是构建成功。

## 4. 前提二：包可见性与登录凭据

GHCR 上的包有自己的可见性，与代码仓库的可见性不完全联动。首次推送后请确认一次。

| 包的可见性       | 目标机器需要什么                                                                                                                 |
|-------------|--------------------------------------------------------------------------------------------------------------------------|
| public (推荐) | 无需任何凭据，直接 <code>docker pull</code> 。私有化部署场景下这是最省事的                                                                       |
| private     | 必须先登录： <code>docker login ghcr.io -u 你的GitHub用户名</code> ，密码填一个带 <code>read:packages</code> 权限的访问令牌（注意： <b>不是账号登录密码</b> ） |

### 1 把包改成 public

打开 <https://github.com/users/mijunyi/packages/container/navi-homepage/settings>，拉到最下方 **Danger Zone** → **Change package visibility**，选 Public 并确认。

**怎么判断包当前是不是 private？** 在一个没登录过 ghcr.io 的机器上执行 `docker pull ghcr.io/mijunyi/navi-homepage:edge`：报 `unauthorized` 或 `denied` 就是 private（或不存在），能正常下载就是 public。

**令牌用错会很迷惑：**「访问令牌」不是登录密码。如果目标机器的 Docker 里 残留着一份过期的 ghcr.io 凭据，`pull` 会报 `unauthorized`，而你会以为是包不可见。可用 `docker logout ghcr.io` 先清掉再重试。

## 5. 前提三：网络能到得了镜像仓库

ghcr.io 的连通性在国内并不稳定，而且大多数国内 Docker 镜像加速器 只加速 Docker Hub，对 ghcr.io 无效。这是本方案最常见的失败点，请提前确认。

### 1 先测一下通不通

```
curl -sS -o /dev/null -w "http_code=%{http_code} time=%{time_total}s\n" \
--max-time 20 https://ghcr.io/v2/
```

返回 401 是正常的（说明连接成功、只是需要认证）；超时或连接被重置才是不通。

### 2 不通的话，三种对策

| 对策                  | 做法                                                                                                                                                                                                                                                                         |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ① 换 GHCR 镜像站<br>最省事 | 编排文件支持用环境变量覆盖镜像地址，不用改文件：<br><code>NAVI_IMAGE=镜像站域名/mijunyi/navi-homepage docker compose -f docker-compose.image.yml up -d</code><br>或写进 <code>.env</code> ： <code>NAVI_IMAGE=镜像站域名/mijunyi/navi-homepage</code><br>镜像站的可信度与可用性请自行确认——它会看到你拉取的镜像内容。                         |
| ② 给 Docker 配代理      | 宿主机有可用的 HTTP 代理时（ <code>http://192.168.1.10:7890</code> 之类）：<br><code>/etc/systemd/system/docker.service.d/http-proxy.conf</code> 写入<br><code>Environment="HTTPS_PROXY=http://代理地址:端口"</code> ，然后 <code>systemctl daemon-reload &amp;&amp; systemctl restart docker</code> |
| ③ 改走跨机导入<br>断网也能用   | 在一台能联网的机器上 <code>docker pull</code> 后导出成文件： <code>docker save ghcr.io/mijunyi/navi-homepage:edge -o navi.tar.gz</code> ，拷到目标机器 <code>docker load -i navi.tar.gz</code> 。详见飞牛 fnOS 部署指南的「跨机导入镜像」一节。<br><b>注意：源机器与目标机器的 CPU 架构必须一致。</b>                                      |

## 6. 前提四：CPU 架构与容器运行时

### CPU 架构

工作流同时构建两种架构，正常情况 Docker 会自动挑对的那一份。先确认目标机器是哪一种：

```
uname -m      # x86_64 → amd64 ; aarch64 / arm64 → arm64
```

| 目标平台                          | 架构          | 说明   |
|-------------------------------|-------------|------|
| 常见 x86 PC / 服务器 / 群晖 x86 机型   | linux/amd64 | 默认支持 |
| 树莓派 4/5、ARM 开发板、Apple Silicon | linux/arm64 | 默认支持 |

### x86\_64 设备：额外确认两点

x86\_64（`uname -m` 输出 `x86_64`）是原生支持的：拉取时 Docker 会自动选中镜像里的 `linux/amd64` 那一份，不需要加 `--platform`，也不存在任何指令模拟开销。但下面两点要先确认，它们和镜像无关，属于宿主机/CPU 层面的硬约束：

| 检查项      | 要求与自检方式                                                                                                                                                                                     |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 必须是 64 位 | 本项目只提供 <code>amd64</code> ，没有 32 位 i386 镜像（Node 22 也没有 32 位 Linux 官方构建）。跑 <code>uname -m</code> ：输出 <code>x86_64</code> 才是 64 位；若输出 <code>i386</code> / <code>i686</code> 则无法部署，只能换 64 位设备。 |

|                       |                                                                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>CPU 需支持 SSE4.2</b> | 这是 Node 运行时 (V8 引擎) 自 Node 14 起的硬性要求, 与镜像本身无关。<br>自检: <code>grep -o sse4_2 /proc/cpuinfo</code> , <b>有输出</b> 即支持。绝大多数 2008 年之后的机器都满足。 |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------|

**老机器上容器「启动即退出」, 日志里是 Illegal instruction (非法指令)**: 说明 CPU 缺少 SSE4.2。典型是 2008 年前的 Intel (Nehalem 之前)、2011 年前的 AMD (Bulldozer 之前), 少数低配 VPS 或未开启 CPU 特性透传的虚拟机也会出现。这类机器跑不了 Node 22 运行时, **只能更换设备**, 或自行用更旧版本的 Node 构建镜像。

**报 no matching manifest for linux/arm64 或容器日志里出现 exec format error**: 说明架构不匹配。前者是镜像里没有这个架构的清单 (检查工作流的 platforms 是否被改过), 后者是拉到/导入成了错误架构的镜像 (常见于跨机导入)。

## 容器运行时

| 运行时                   | 要求                                                                                                                  |
|-----------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Docker CE (推荐)</b> | 20.10 及以上。要跑 <code>docker compose -f ...</code> 需 Docker Compose v2 ( <code>docker compose version</code> 能正常输出版本号) |
| Podman                | 可用。compose 部分改用 <code>podman-compose</code> , 或直接用第 13 节的 <code>podman run</code> 等价命令                              |
| containerd / nerdctl  | 可用, 把下文命令里的 <code>docker</code> 换成 <code>nerdctl</code> 即可                                                          |

镜像基于 `node:22-alpine`, 体积小、无额外依赖; 宿主机**不需要安装 Node.js**, 也不需要 npm。

## 7. 部署步骤 (命令行, 推荐)

### 1 建立目录并导出初始配置

```
# 选一个位置, 示例用 /opt/navi
mkdir -p /opt/navi/data
cd /opt/navi

# 从镜像里直接导出示例配置 — 只依赖镜像仓库, 不需要能访问 github.com
docker run --rm ghcr.io/mijunyi/navi-homepage:latest \
  cat /app/data/config.json > data/config.json

# 确认导出来了 (应是一个几百字节到几 KB 的 JSON)
ls -l data/config.json
```

如果拉取用的是其它标签, 上面的 `:latest` 请也跟着换 (例如只跑过 main 分支的工作流, 那就用 `:edge`)。

### 2 准备 .env (放访问密码)

```
cat > .env <<'EOF'
NAVI_PASSWORD=换成一个足够强的密码
# 可选: 固定版本, 避免 latest 漂移
# NAVI_TAG=1.0.0
# 可选: 国内换用 GHCR 镜像站
# NAVI_IMAGE=镜像站域名/mijunyi/navi-homepage
EOF

chmod 600 .env          # 密码文件不要给别人读
```

**想更彻底一点** (连 `.env` 里都不放明文密码): 改用哈希。注意服务端比对的是 `sha256("navi-site:" + 密码)`, 必须带 `navi-site:` 前缀, 用裸 `sha256` 会永远登录失败且不报错:

```
node -e "console.log(require('crypto').createHash('sha256').update('navi-site:'+process.argv[1]).digest('hex'))" '你的密码'
```

然后把它写成 `NAVI_PASSWORD_HASH=...`, 并删掉 `NAVI_PASSWORD` 那一行。

### 3 拿到编排文件

把源码仓库里的 `docker-compose.image.yml` 放到 `/opt/navi/`:

```
# 有源码仓库时直接拷; 没有的话从网页下载这一个文件即可
# https://github.com/mijunyi/navi-homepage/blob/main/docker-compose.image.yml
cp /path/to/repo/docker-compose.image.yml .
```

### 4 先 pull, 再启动

```
docker compose -f docker-compose.image.yml pull
docker compose -f docker-compose.image.yml up -d
```

**这步顺序不能省。**直接 `up -d` 也能起, 但 Docker 会优先用本地已有的同名镜像——升级场景下你会以为更新了, 其实跑的还是旧的。

### 5 确认起来了

```
docker compose -f docker-compose.image.yml ps
docker logs --tail 30 navi
curl -s http://127.0.0.1:8080/api/health
```

健康接口应返回形如 `{"ok":true,"ipv6":true,"authEnabled":true}`。看到 `"authEnabled":true` 说明密码保护已生效。

## 8. 部署步骤 (图形界面 / NAS)

在飞牛 fnOS、群晖等 NAS 上，用图形界面创建「Compose 项目」更顺手。要点如下：

### 1 把 docker-compose.image.yml 的内容粘进编辑器

注意 NAS 的图形界面通常不支持构建——而这正好是纯拉取编排的强项：它本来就不需要构建，所以比源码构建模式在 NAS 上更顺。

### 2 记住两条 YAML 铁律

- 同级 `-` 必须对齐在同一列；取消注释时最常犯的错就是多删或少删了空格。
- 同时改多行时，别只改一部分——**缩进比同级深一格不会报错**，而是被折进上一条，结果是变量静默失效，部署后表现为「设了却不生效」，极难排查。

改完可用项目里的自检脚本在线下验一遍（它会解析出真正生效的 environment 并标出被折叠的值）：

```
node scripts/check-compose.cjs docker-compose.image.yml
```

### 3 数据目录要指向宿主机真实路径

图形界面里 `./data` 可能被解释到别处。建议直接写成绝对路径，并先把 config.json 放进去：

```
- /vol1/1000/docker/navi/data:/app/data
```

### 4 密码变量怎么给

图形界面一般没有 .env 的概念，两种做法：把 `${NAVI_PASSWORD:?...}` 整行替换成 `- NAVI_PASSWORD=你的密码`（注意别把这个文件提交到任何仓库），或者把 .env 和编排文件放同一目录（多数 NAS 的 Compose 项目会读取同目录 .env）。

### 5 容器属主问题

容器以 root 运行，写入 data/ 的文件属主会是 root:root，NAS 的文件管理器（通常是 UID 1000）可能改不动。需要时执行：

```
chown -R 1000:1000 /vol1/1000/docker/navi/data
```

## 9. 首次访问与验证

| 检查项  | 命令                                                    | 预期                                     |
|------|-------------------------------------------------------|----------------------------------------|
| 容器在跑 | <code>docker ps   grep navi</code>                    | 状态 Up                                  |
| 健康接口 | <code>curl -s http://127.0.0.1:8080/api/health</code> | <code>"ok":true</code>                 |
| 需要登录 | 浏览器打开 <code>http://宿主机IP:8080</code>                  | 跳转到登录页                                 |
| 数据落盘 | <code>ls -l data/</code>                              | 保存一次后 <code>config.json</code> 的修改时间更新 |



|          |                                                    |       |
|----------|----------------------------------------------------|-------|
| 确认跑的是新代码 | node scripts/check-deploy.cjs<br>http://IP:8080 密码 | 退出码 0 |
|----------|----------------------------------------------------|-------|

登录后进入编辑模式，任意改一处再保存，然后检查 data/config.json 是否有变化——这能一次性验证「挂载正确 + 权限可写」，比看日志更直接。

## 10. 升级、回滚与固定版本

### 升级到新版

```
cd /opt/navi
docker compose -f docker-compose.image.yml pull
docker compose -f docker-compose.image.yml up -d
```

data/ 是路径挂载，Docker 不管它的生命周期，**重建容器不会动你的导航数据与图片**。想更保险就先备份：

```
tar czf navi-data-$(date +%F).tar.gz -C ./data .
```

### 固定版本（生产环境推荐）

`latest` 会随每次发版移动，多台机器之间容易出现版本不一致。建议在 .env 里钉住版本号：

```
NAVI_TAG=1.0.0
```

### 回滚

```
# 把 NAVI_TAG 改回上一个版本号，然后
docker compose -f docker-compose.image.yml pull
docker compose -f docker-compose.image.yml up -d
```

回滚只换程序，**不会**回滚数据。如果新版改过配置结构，回滚前请用 页面里的「备份」功能导出一份旧数据。

### 彻底卸载

```
docker compose -f docker-compose.image.yml down
# 镜像可以删；data/ 目录请自行确认后再删（那是你唯一的真实数据）
docker rmi ghcr.io/mijunyi/navi-homepage:latest
```

## 11. 常见问题与排查

| 现象 | 原因 | 处理 |
|----|----|----|
|----|----|----|

|                                      |                                                    |                                                                 |
|--------------------------------------|----------------------------------------------------|-----------------------------------------------------------------|
| manifest unknown / not found         | 镜像还没发布过（工作流一次都没跑成功）                                | 去 Actions 手动触发一次，见第 3 节                                         |
| unauthorized / denied                | 包是 private；或本机残留了过期的 ghcr 凭据                       | 把包改成 public；或 docker logout ghcr.io 后重新登录                       |
| dial tcp ... i/o timeout             | 网络到不了 ghcr.io                                      | 第 5 节三种对策：镜像站 / 代理 / 跨机导入                                       |
| no matching manifest for linux/arm64 | 镜像里没有该架构的清单                                        | 确认工作流 platforms 仍含 linux/amd64,linux/arm64                      |
| 容器日志 exec format error               | 镜像架构与宿主机不符（多为跨机导入导致）                               | 重新拉取对应架构，或改用同架构机器导出                                             |
| no matching manifest for linux/386   | 目标机是 32 位 x86 (uname -m 为 i386 / i686)，项目没有对应镜像    | 无解：需更换 64 位设备（Node 22 亦无 32 位 Linux 构建）                         |
| 容器启动即退出，日志 Illegal instruction       | CPU 缺少 SSE4.2 指令集（约 2008 年前的 Intel / 2011 年前的 AMD） | 先跑 grep -o sse4_2 /proc/cpuinfo 确认；确为老 CPU 则换设备，或自行用旧版 Node 构建  |
| 容器能起来，但保存配置报权限错误 / 数据写不进去            | RHEL / CentOS / Fedora 等发行版开启了 SELinux，挂载未授权容器写入   | 挂载末尾加 :Z，即 ./data:/app/data:Z                                   |
| compose 报 NAVI_PASSWORD is required  | 没有提供密码变量                                           | 补 .env 或在编排文件里显式写 NAVI_PASSWORD=                                |
| 容器启动后立刻退出，日志有 ENOENT                 | 宿主机 data/config.json 不存在（挂载遮住了镜像内置示例）              | 执行第 7 节第 1 步导出配置                                                |
| 页面能打开，但点「保存」失败                       | 挂载方式不是整目录（图形界面可能改写成单文件挂载）                          | 改回 ./data:/app/data，不要挂单个 config.json                           |
| 改了代码/重建了容器，页面还是老样子                   | 浏览器强缓存                                             | 先 Ctrl+F5；仍不对再跑 node scripts/check-deploy.cjs 定性                |
| pull 完还是旧版本                          | Docker 复用了本地镜像，或 compose 缓存了镜像 ID                  | docker compose -f docker-compose.image.yml down 后再 pull + up -d |
| 工作流红了，提示 invalid reference format    | GHCR 镜像名必须全小写                                      | 工作流里有小写转换步骤，检查它是否被删掉                                            |
| 工作流里登录 ghcr.io 失败                    | 缺少 packages: write 权限                              | 确认工作流顶部的 permissions 仍是 packages: write                         |

## 12. 部署检查清单

- ☐ 目标机器能 curl 通 https://ghcr.io/v2/ (返回 401 也算通)
- ☐ 确认过包可见性 (public 无需登录, private 需 docker login)
- ☐ 确认过 CPU 架构: uname -m 输出 x86\_64 或 aarch64 (不能是 32 位的 i386 / i686)
- ☐ x86 机器额外确认 CPU 支持 SSE4.2: grep -o sse4\_2 /proc/cpuinfo 有输出
- ☐ data/config.json 已在宿主机就位
- ☐ .env 里有 NAVI\_PASSWORD (或哈希), 且文件权限为 600
- ☐ 用的是「整目录挂载」./data:/app/data, 不是单个 config.json
- ☐ 执行过 docker compose -f docker-compose.image.yml pull
- ☐ docker compose -f docker-compose.image.yml up -d 后 /api/health 返回 "ok":true
- ☐ 浏览器能打开并跳转登录页, 登录后保存一次确认数据真的落盘
- ☐ (生产) 已把 NAVI\_TAG 固定到具体版本号
- ☐ 已做过一次数据备份: tar czf navi-data-\$(date +%F).tar.gz -C ./data .

## 13. 命令速查

```
# 查看目标机器架构 (须为 x86_64 或 aarch64, 不能是 32 位的 i386/i686)
uname -m

# x86 机器: 确认 CPU 支持 SSE4.2 (无任何输出则跑不了 Node 22 运行时)
grep -o sse4_2 /proc/cpuinfo

# 测试镜像仓库连通性 (401 = 通)
curl -sS -o /dev/null -w "%{http_code}\n" --max-time 20 https://ghcr.io/v2/

# 首次: 建目录 + 导出初始配置
mkdir -p /opt/navi/data && cd /opt/navi
docker run --rm ghcr.io/mijunyi/navi-homepage:latest cat /app/data/config.json > data/config.json

# 启动 / 升级
docker compose -f docker-compose.image.yml pull
docker compose -f docker-compose.image.yml up -d

# 看状态与日志
docker compose -f docker-compose.image.yml ps
docker logs --tail 50 navi

# 健康检查
curl -s http://127.0.0.1:8080/api/health

# 换镜像站 / 固定版本 (也可写进 .env)
NAVI_IMAGE=镜像站域名/mijunyi/navi-homepage NAVI_TAG=1.0.0 \
  docker compose -f docker-compose.image.yml up -d
```

```
# 备份数据
tar czf navi-data-$(date +%F).tar.gz -C ./data .

# 回滚到指定版本
NAVI_TAG=1.0.0 docker compose -f docker-compose.image.yml pull
NAVI_TAG=1.0.0 docker compose -f docker-compose.image.yml up -d

# 不用 compose 的等价写法
docker run -d --name navi --restart unless-stopped \
  -p 8080:80 \
  -e NAVI_PASSWORD=你的密码 \
  -e NAVI_CONFIG_PATH=/app/data/config.json \
  -e NAVI_UPLOAD_DIR=/app/data/uploads \
  -v /opt/navi/data:/app/data \
  ghcr.io/mijunyi/navi-homepage:latest

# 卸载（保留数据目录）
docker compose -f docker-compose.image.yml down

# 收集排查信息
docker compose -f docker-compose.image.yml config
```

**相关文档：** 源码构建与通用 Docker 说明见 <docs/docker-guide.html> ； 飞牛 fnOS 专有差异（存储路径、图形界面限制、文件属主、跨机导入）见 <docs/fnos-deploy-guide.html> 。